

DS ASSIGNMENT

Great — concise, textbook-style answers incoming. I'll keep explanations clear, formal, and human-sounding, with short C code snippets where requested. I'll label sections for Unit 1 and Unit 2 and answer each question in order.

Unit 1

1. Differentiate between a normal variable and a pointer variable.

- **Normal variable:** Directly stores a value (e.g., `int x = 10;`). Memory holds the value itself.
 - **Pointer variable:** Stores the memory address of another variable (e.g., `int *p = &x;`). Memory holds an address; you must dereference (`*p`) to access the target value.
 - **Key differences:**
 - Type: `int` vs `int*`.
 - Value: actual data vs address of data.
 - Use: normal for data storage; pointer for indirection, dynamic memory, arrays, and function callbacks.
 - Size: pointer size is platform-dependent (typically 4 or 8 bytes), independent of pointed type's size.
-

2. What is the use of `&` and `*` in pointers? Illustrate with a simple C program.

- `&` (address-of): yields the address of a variable.
- `*` (dereference): accesses the value stored at the address (when used on a pointer).
- also declares pointer types: `int *p;`

Example C program

```
#include <stdio.h>

int main() {
    int a = 42;
    int *p = &a;    // & gives address of 'a'
    printf("a = %d\n", a);
    printf("address of a = %p\n", (void*)&a);
    printf("p holds = %p\n", (void*)p);
    printf("value at p = %d\n", *p); // *p dereferences
    *p = 10;        // modify a through pointer
    printf("a after modification = %d\n", a);
    return 0;
}
```

3. Program to find sum of elements in an array using pointers.

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    int *p = arr; // points to first element
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        sum += *(p + i); // pointer arithmetic
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

4. C program to swap two numbers using pointers and a function.

```

#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;
    printf("Before: a=%d, b=%d\n", a, b);
    swap(&a, &b);
    printf("After: a=%d, b=%d\n", a, b);
    return 0;
}

```

5. Explain pointer arithmetic with examples.

- Pointer arithmetic adjusts the pointer by multiples of the size of the pointed type.
- If `p` is `T*`, then `p + 1` advances by `sizeof(T)` bytes.
- Example:

```

int arr[3] = {10,20,30};
int *p = arr; // points to arr[0]
p + 1;        // points to arr[1]
*(p + 2);     // arr[2] == 30

```

- Valid ops: `p + n`, `p - n`, `p1 - p2` (gives element difference). Comparison operations (`<`, `>`) are meaningful only when pointers belong to the same array.

6. Purpose and short syntax for pointer types

i. Null pointer

- Purpose: Indicates pointer points to nothing. Safe sentinel.
- Syntax: `int *p = NULL;`

ii. Dangling pointer

- Purpose: (Bad) pointer referring to memory already freed or out-of-scope. Avoid by setting to `NULL` after `free` or not returning pointer to local stack.
- Example cause:

```
int *f() {  
    int x = 5;  
    return &x; // dangling - x goes out of scope  
}
```

iii. Void pointer

- Purpose: Generic pointer type; can hold address of any data type; requires cast to dereference.
- Syntax: `void *vp = malloc(100);`

iv. Wild pointer

- Purpose: Uninitialized pointer containing garbage address — dangerous. Always initialize pointers.
- Example: `int *p; /* wild until assigned */`

v. Double pointer

- Purpose: Pointer to pointer, used for dynamic arrays, functions that modify pointer arguments, or arrays of pointers.
- Syntax: `int **pp;`

vi. Function pointer

- Purpose: Holds address of a function; used for callbacks or dynamic dispatch.
 - Syntax: `int (*fp)(int, int);` and assignment `fp = &myFunc;`
-

7. What is dynamic memory allocation? How is it different from static memory allocation?

- **Dynamic memory allocation:** Requesting memory at runtime (heap) using `malloc`, `calloc`, `realloc`, and releasing with `free`. Flexible, lifetime controlled by programmer.
- **Static memory allocation:** Allocation that occurs at compile/link/load time or on stack (e.g., local arrays, global vars). Lifetime tied to scope or program lifetime.
- **Differences:**
 - **When:** dynamic at runtime; static at compile/load or function entry (stack).
 - **Lifetime:** dynamic until `free`; static lifetime until scope exit or program end.
 - **Size:** dynamic can be variable; static often fixed (unless VLAs).
 - **Management:** dynamic requires explicit management; static managed by language runtime.

8. Explain the four dynamic memory allocation functions in C.

- `malloc(size_t size)` — allocate `size` bytes, returns `void*`. Contents uninitialized.
- `calloc(size_t nmem, size_t size)` — allocate `nmem * size` bytes and initialize all bytes to 0.
- `realloc(void *ptr, size_t size)` — resize previously allocated block `ptr` to `size` bytes; may move block and return new pointer.
- `free(void *ptr)` — deallocate memory previously allocated by `malloc/calloc/realloc`. After `free`, pointer becomes invalid (set to `NULL` to be safe).

9. Short program: allocate memory dynamically for an array using `malloc()`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
```

```
int n;
printf("Enter number of elements: ");
if (scanf("%d", &n) != 1) return 1;
int *arr = (int*) malloc(n * sizeof(int));
if (!arr) {
    fprintf(stderr, "Memory allocation failed\n");
    return 1;
}
for (int i = 0; i < n; ++i) arr[i] = i + 1;
for (int i = 0; i < n; ++i) printf("%d ", arr[i]);
printf("\n");
free(arr);
arr = NULL;
return 0;
}
```

10. Draw taxonomy of various Data Structures.

(Concise textual taxonomy)

- **Primitive:** int, float, char, bool
- **Non-primitive**
 - **Linear**
 - Arrays
 - Linked Lists (singly, doubly, circular)
 - Stacks
 - Queues (simple, circular, priority)
 - **Non-linear**
 - Trees (binary tree, BST, AVL, red-black, heap, trie)
 - Graphs (directed, undirected, weighted)
 - **Hashing**

- Hash table / map / dictionary
 - **Advanced/Derived**
 - Sets, multisets
 - Priority queues (heaps)
 - Disjoint Set (Union-Find)
 - **Persistent / External:** B-tree, B+ tree, LSM-tree
-

Unit 2

1. Algorithm (and concise C program idea) to convert infix to postfix

Algorithm (Shunting-yard / stack-based):

1. Initialize an empty stack for operators and an output list.
2. Scan infix token by token:
 - If token is operand → append to output.
 - If token is '(' → push to stack.
 - If token is ')' → pop from stack to output until '(' encountered; pop '('.
 - If token is operator `op1`:
 - While stack top is operator `op2` and (`precedence(op2) > precedence(op1)` or `precedence(op2) == precedence(op1)` and `op1` is left-associative), pop `op2` to output.
 - Push `op1` to stack.
3. After input scanned, pop remaining operators to output.
4. Output is postfix.

Notes: handle operator precedence (`^` highest, typically right-associative, then `*`, `/`, `%`, then `+`, `-`), associativity (exponent `^` is usually right-associative).

2. Convert infix to postfix (show final postfix; tracing omitted for brevity)

i) $A + B * C - (D / E ^ F) * G - H$

- Precedence: $^ > / \% > +$.
- Postfix: $A B C * + D E F ^ / G * - H -$

ii) $A * (B - C / D) + G \% F$

- Postfix: $A B C D / - * G F \% +$

iii) $p - q / r * (s ^ t + u)$

- Postfix: $p q r / s t ^ u + * -$

iv) $m + n ^ p \% q * (u - v / w)$

- Interpret $\%$ precedence same as $/$ (in C). $^$ here is exponent in math (if so, higher than $\%$).
- Postfix: $m n p ^ q \% u v w / - * +$

(If $\%$ is modulo with same precedence as $*$ then ordering is as written above.)

3. Algorithm for evaluating a valid postfix expression

1. Create an empty stack.
2. For each token in postfix expression:
 - If token is operand $\rightarrow$ push its numeric value onto the stack.
 - If token is operator op :
 - Pop operand2 (top), pop operand1 (next).
 - Compute $result = operand1 op operand2$.
 - Push $result$.
3. After processing tokens, stack should have one value $\rightarrow$ result.

(Handle division carefully; for integers use integer division; for floats use float arithmetic.)

4. Evaluate postfix expressions (show final numeric result and brief trace)

i) $4\ 5\ 6\ *\ +\ 2\ 4\ 3\ ^\ /\ -$

- Evaluate stepwise:
 - $5\ 6\ * = 30 \rightarrow$ stack: $4\ 30$
 - $4\ 30\ + = 34 \rightarrow$ stack: 34
 - Next $2\ 4\ 3\ ^ \rightarrow 3^ = 3^?$ interpret $4\ 3\ ^ = 4^3 = 64 \rightarrow$ push $2\ 64\ /$
 - $2\ 64\ / = 2 / 64 = 0.03125$ (float)
 - $34\ 0.03125\ - = 33.96875$
- **Result:** 33.96875

ii) $6\ 5\ 4\ *\ 2\ /\ +\ 7\ 3\ ^\ -$

- $5\ 4\ * = 20 \rightarrow$ stack $6\ 20$
- $20\ 2\ /\ = 10 \rightarrow$ stack $6\ 10$
- $6\ 10\ + = 16 \rightarrow$ stack 16
- $7\ 3\ ^ = 343 \rightarrow$ stack $16\ 343$
- $16\ 343\ - = -327$
- **Result:** 327

iii) $a\ b\ /\ c\ *\ d\ e\ ^\ -\ f\ +$ with $a=12, b=3, c=5, d=15, e=4, f=20$

- $a\ b\ /\ = 12/3 = 4$
 - $4\ c\ * = 4 * 5 = 20$
 - $d\ e\ ^ = 15^4 = 50625$
 - $20\ 50625\ - = -50605$
 - $50605\ f\ + = -50605 + 20 = -50585$
 - **Result:** 50585
-

5. Algorithm to convert infix to prefix and convert two expressions

Algorithm (common method):

1. Reverse the infix expression (swap (with)).
2. Replace each (with) and vice versa in the reversed string.
3. Convert the result to postfix using infix→postfix algorithm.
4. Reverse the postfix to get prefix.

Conversions:

i) $a + b / c - d * e$

- Infix: $a + b / c - d * e$
- Postfix (intermediate): $a b c / + d e * -$
- Prefix: $+ a / b c * d e$

So **Prefix:** $- + a / b c * d e$

ii) $a * (b + c \% d) - e / f$

- $\%$ same precedence as $/$; inside parentheses $b + (c \% d)$.
- Postfix: $a b c d \% + * e f / -$
- Prefix: $* a + b \% c d / e f$

So **Prefix:** $- * a + b \% c d / e f$

6. Define recursion. List important features of a recursive program.

- **Recursion:** A function that calls itself directly or indirectly to solve a problem by breaking it into smaller subproblems of the same form.
- **Important features:**
 - **Base case(s):** condition(s) where recursion stops.
 - **Recursive case:** function calls itself with smaller/simpler arguments.
 - **Progress:** each recursive call must move toward a base case.

- **Stack usage:** each call uses stack frame; excessive depth can cause stack overflow.
- **Overhead:** recursive calls have overhead vs iterative solutions sometimes.
- **Elegance:** recursion can simplify expression of divide-and-conquer algorithms (e.g., quicksort, mergesort, tree traversals).

7. Recursive and iterative methods to solve Tower of Hanoi

Problem: Move n disks from source peg A to target peg C using auxiliary peg B; larger disks cannot be placed on smaller ones.

Recursive algorithm (standard):

```
Hanoi(n, source, aux, target):
  if n == 1:
    move disk from source to target
  else:
    Hanoi(n-1, source, target, aux)
    move disk from source to target
    Hanoi(n-1, aux, source, target)
```

Time complexity: $T(n) = 2T(n-1) + 1 \rightarrow O(2^n)$ moves; exact moves = $2^n - 1$.

Iterative method (non-recursive):

- Use the fact minimal solution alternates moves and pattern depends on parity of n .
- Use a stack or simulate using iterative loop, performing legal moves between pegs in correct sequence. Common iterative approach:
 - If n is even, swap target and auxiliary labels.
 - Repeat $2^n - 1$ times:
 - On odd moves: move smallest disk to the next peg in prescribed order.
 - On even moves: make the only legal move not involving the smallest disk.

- Iterative solution achieves same move count but is more complex to implement.

8. Evaluate prefix expressions with stack tracing (final result shown)

i) $+ / 4 7 * 5 2$

Interpretation: $+ (/ 4 7) (* 5 2)$

- $/ 4 7 = 4 / 7 \approx 0.5714286$
- $5 2 = 10$
- $+ \rightarrow 0.5714286 + 10 = 10.5714286$

Result: ≈ 10.5714286

ii) $+ / 8 ^ 4 3 * 9 5$

Interpretation: $+ (/ 8 ^ 4 3) (* 9 5)$

- $^ 4 3 = 4 ^ 3 = 64$
- $/ 8 64 = 0.125$
- $9 5 = 45$
- $+ \rightarrow 45.125$

Result: 45.125

(If integer division is intended, use integer arithmetic accordingly.)

9. Classification of queue data structure. List applications.

Classification:

- **Simple (linear) queue** — FIFO via array or linked list.
- **Circular queue** — reuses freed space by circular indexing.
- **Deque (double-ended queue)** — insert/remove from both ends.
- **Priority queue** — elements removed based on priority, not strict FIFO (often implemented with heaps).

- **Concurrent/Blocking queue** — thread-safe variants used in multi-threading.

Applications:

- CPU scheduling (ready queue), breadth-first search (BFS) in graphs.
- IO buffering (print spoolers, streaming).
- Producer-consumer problems (message queues).
- Level-order traversal of trees.
- Networking (packet buffers), task scheduling, event handling.

10. What is circular queue? C implementation using array.

Concept: Circular queue (ring buffer) treats array as circular; front and rear wrap using modulo `capacity`. Avoids moving elements after dequeue and uses all array slots efficiently.

C implementation (concise):

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

typedef struct {
    int arr[MAX];
    int front, rear;
} CircularQueue;

void init(CircularQueue *q) {
    q->front = -1;
    q->rear = -1;
}

int isEmpty(CircularQueue *q) {
    return q->front == -1;
}
```

```

int isFull(CircularQueue *q) {
    return ((q->rear + 1) % MAX) == q->front;
}

void enqueue(CircularQueue *q, int val) {
    if (isFull(q)) {
        printf("Queue is full\n");
        return;
    }
    if (isEmpty(q)) q->front = 0;
    q->rear = (q->rear + 1) % MAX;
    q->arr[q->rear] = val;
}

int dequeue(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1; // sentinel
    }
    int val = q->arr[q->front];
    if (q->front == q->rear) { // single element removed
        q->front = q->rear = -1;
    } else {
        q->front = (q->front + 1) % MAX;
    }
    return val;
}

void display(CircularQueue *q) {
    if (isEmpty(q)) {
        printf("Queue empty\n");
        return;
    }
    int i = q->front;
    while (1) {

```

```
    printf("%d ", q→arr[i]);  
    if (i == q→rear) break;  
    i = (i + 1) % MAX;  
}  
printf("\n");  
}  
  
/* Example main can be added to test enqueue/dequeue */
```